Recovering coordinates from paper maps

Gabriel Arellano

2026-11-06

Table of contents

1 Overview **1**

2 Obtain clicked coordinates **2**

3 Homography and affine transformations **3**

4 Historical CTFS bug fix **5**

1 Overview

This tutorial covers the process of recovering global plot coordinates from a scanned paper map. The process has two steps:

- First, click labelled points and controls;
- then, use homography (preferred) or affine transformation (plan B).

The old CTFS workflow required using imageJ. The current version of the code includes a native R point clicker with minimal dependency (only requires base and `magick` R package).

As the old CTFS algorithm, the homography/affine transformations assume a planar map. They do not repair folds, tears or errors in the original field mapping. Previous simulations (see "evaluation" folder) have demonstrated that these algorithms have low error in practice: lower than old CTFS algorithm, and lower than mapping/clicking human errors.

Replace the example full paths in each section with your own. No Git repository, particular working directory or shared setup objects are required.

2 Obtain clicked coordinates

There are different ways to obtain the clicked coordinates in a scanned paper map, like ImageJ or Fiji. The historical CTFS workflow used a modified PointPicker plugin. Here we use a native R alternative.

Run the clicker code in the R console, not while rendering. The input map can be in any format that `magick::image_read()` can read. On macOS, uncomment `quartz()` if you need a separate graphics window.

Click a point, type its label in the R console, and press Enter. I recommend keyboard shortcuts to go from one window to the other. Press Esc or Finish when done. The clicker saves columns `id`, `x` and `y` after every point.

To follow the transformation example with the supplied `example-map-clicked-coor.csv`, skip this chunk. To click the map yourself, save to a new CSV, then use that file in the next section. The clicker refuses to overwrite existing CSVs.

Unlike old CTFS algorithm, homography and affine transformations can leverage more than four control points, using least squares fit when needed. Thus, you can store as many control points as practical: four corners, and anything else you trust. Label them carefully in ways that you can use later to differentiate individual control points. In our example, I will label control points as the paper map shows: four corners as A, B, C and D, and five additional control points as C0, C1, C2, C3 and C4.

```
# Full paths to the georeference code, the input map, and where to store output.
# (Update as needed):

path.to.fgeo2 = "/path/to/fgeo2/"

path.to.code = paste0(path.to.fgeo2, "06_topography_habitat/R/new/georeference.R")
path.to.map = paste0(path.to.fgeo2, "data_examples/example-map.jpeg")
path.to.csv = paste0(path.to.fgeo2, "data_examples/my-clicked-coor.csv")

# Load and run georeferencing code:
source(path.to.code)

# grDevices::quartz(width = 8, height = 8) # Optional on macOS
clicked <- point_clicker(
    image.path = path.to.map,
    csv.path = path.to.csv
)
```

3 Homography and affine transformations

Homography is the recommended method. It can correct perspective in an angled photo of a flat map. It requires four reliable control points, though. Affine transformation only needs three control points and it is the plan B for when a corner is missing or unusually unreliable. See the corresponding evaluation file for more details.

This section reads the clicked CSV directly. It does not require the clicker to have been run in the same R session. The within-image coordinates could have been obtained by imageJ or other methods too.

The code below implements these steps:

1. reads code and the CSV with within-image coordinates;
2. splits coordinates into control points vs. tree locations;
3. declare the known coordinates of the control points, relative to the whole plot, as in the paper map;
4. runs the transformations to infer the tree locations relative to the whole plot.

```
# Full paths to the georeference code and clicked coordinates.
# (Update as needed):

path.to.fgeo2 = "/path/to/fgeo2/"

path.to.code = paste0(path.to.fgeo2, "06_topography_habitat/R/new/georeference.R")
path.to.csv = paste0(path.to.fgeo2, "data_examples/example-map-clicked-coor.csv")

# Load code and clicked coordinates:
source(path.to.code)

clicked <- read.csv(path.to.csv, colClasses = c(id = "character"))
stopifnot(!anyNA(clicked$id), all(nzchar(clicked$id)), !anyDuplicated(clicked$id))

head(clicked, 15) # we should see all controls and some trees


# Split the set of clicked coordinates into controls vs. trees
control.ids <- c("A", "B", "C", "D", "C0", "C1", "C2", "C3", "C4") # in my example

controls <- clicked[which(clicked$id %in% control.ids),]
trees <- clicked[which(!clicked$id %in% control.ids),]

# Declare known / true locations of the control points
```

```r
# (usually pre-defined by convention, from quadrat-level info, from the map itself...)

known.controls <- rbind(
  data.frame(id = "A", x = 20, y = 50),
  data.frame(id = "B", x = 20, y = 60),
  data.frame(id = "C", x = 30, y = 60),
  data.frame(id = "D", x = 30, y = 50),
  data.frame(id = "C0", x = 25, y = 55),
  data.frame(id = "C1", x = 25 , y = 50),
  data.frame(id = "C2", x = 20, y = 55),
  data.frame(id = "C3", x = 25, y = 60),
  data.frame(id = "C4", x = 30, y = 55)
)

# Just keep {x, y}, make sure order is right
rownames(controls) <- controls$id
rownames(known.controls) <- known.controls$id
known.controls <- known.controls[rownames(controls),]
rownames(trees) <- trees$id
controls <- as.matrix(controls[, c("x", "y")])
known.controls <- as.matrix(known.controls[, c("x", "y")])
trees <- as.matrix(trees[, c("x", "y")])


# Homography (recommended method)
homography.xy <- apply_homography_transformation(
    known.xy.of.controls = known.controls,
    clicked.xy.of.controls = controls,
    clicked.xy.of.points = trees
)

#all(rownames(homography.xy) == rownames(trees)) # should be TRUE

# Affine (plan B)
affine.xy <- apply_affine_transformation(
    known.xy.of.controls = known.controls,
    clicked.xy.of.controls = controls,
    clicked.xy.of.points = trees
)

#all(rownames(homography.xy) == rownames(trees)) # should be TRUE
```

```r
# Visualize and compare with paper map:
plot(homography.xy, asp = 1, las = 1,
     xlab = "Global plot x (m)",
     ylab = "Global plot y (m)", col = 0)
text(homography.xy, rownames(homography.xy))

abline(v = c(20, 30), h = c(50, 60))
abline(v = 25, h = 55, lty = 3)

points(known.controls, pch = 20, col = "red")
text(known.controls, rownames(known.controls), col = "red", xpd = TRUE)


# Compare both methods. They should differ minimally in normal conditions.
# In my case, mean absolute difference between methods is <1 cm.
#mean(abs(homography.xy - affine.xy))

# Store results wherever you want
# ...
```

If controls are declared in local coordinates instead of global coordinates, convert the results with `compile_points_from_quadrats()` from the quadrat code, as explained in the quadrat tutorial.

4 Historical CTFS bug fix

The CTFS/ImageJ conversion is not the recommended pipeline and will not be retained in fgeo2. The archived `imageJ.r` implementation remains available with its geometry dependencies. The original workflow is documented in the 2011 CTFS PDF tutorial and HTML tutorial.

The archived `distance.to.side()` passes one-column data frames instead of numeric intersection coordinates. The function below is a replacement that fixes that one bug only. It does not fix inconsistent corner defaults, exterior points folded into a quadrat, or other limitations of the legacy workflow.

This function would work only if the archived CTFS environment is loaded, including utilities that provide `IfElse()`.

```r
# distance.to.side.original <- distance.to.side # to keep both versions

# Fix bug
distance.to.side <- function(pt, target, side1, side2)
{
    perpendicular <- perpendicular.distance(
        b = target[1], m = target[2], x = pt[1], y = pt[2]
    )

    parallel <- drop(as.matrix(parallel.line(
        b = side1[1], m = side1[2], x = pt[1], y = pt[2]
    )))
    intersection <- intersection.of.lines(
        b1 = parallel[1], m1 = parallel[2],
        b2 = target[1], m2 = target[2]
    )
    distance1 <- xydist(pt[1], pt[2], intersection$x, intersection$y)

    parallel <- drop(as.matrix(parallel.line(
        b = side2[1], m = side2[2], x = pt[1], y = pt[2]
    )))
    intersection <- intersection.of.lines(
        b1 = parallel[1], m1 = parallel[2],
        b2 = target[1], m2 = target[2]
    )
    distance2 <- xydist(pt[1], pt[2], intersection$x, intersection$y)

    distances <- c(perpendicular, distance1, distance2)
    c(mean(distances), sd(distances) / mean(distances))
}

stopifnot(isTRUE(all.equal(
    unname(distance.to.side(
        pt = c(5, 5),
        target = c(0, Inf),
        side1 = c(10, 0),
        side2 = c(0, 0)
    )),
    c(5, 0)
)))
```